Github 账号： GlueNa2SiO3

个人博客关于密码学实验的链接:
https://github.com/Glue-Na2SiO3/GLUE

实验题目：密码学第二次实验

实验摘要:

本实验围绕密码学中的四个核心任务展开，包括 ECB 模式的逐字节解密攻击（进阶版）、PKCS#7 填充验证、CBC 模式的比特翻转攻击，以及基于欧洲电子护照机读区信息生成 AES 密钥并解密的实践。通过对提供的代码解析与实验操作，深入理解对称加密算法的工作机制、潜在安全漏洞及密钥生成逻辑，旨在掌握密码学攻击原理与实际应用中的密钥管理方法。

题目描述

1.Byte-at-a-time ECB decryption (Harder)

针对 AES-ECB 模式加密的密文，在已知已知前缀（包含随机前缀和目标字符串）的情况下，通过构造特定明文，利用 ECB 模式相同明文块生成相同密文块的特性，逐字节破解目标字符串。核心代码为 6.py，需解决随机前缀导致的块对齐问题，确定块大小、前缀长度后完成解密。

2.PKCS#7 padding validation

实现并验证 PKCS#7 填充的合法性校验逻辑，判断给定字节流的填充是否符合规范（最后 n 字节均为 n）。核心代码为 7.py，需处理有效填充与无效填充（如填充长度超范围、填充字节不一致）的场景。

3.CBC bit flipping attacks

利用 CBC 模式的加密特性（前序密文块影响后续明文块解密），通过修改特定密文块的比特位，将明文篡改为包含;admin=true;的目标字符串。核心代码为 8.py，需定位前缀占用的块数，构造攻击明文并翻转对应比特。

4.2MTC3 AES key — encoded in the machine readable zone of a European ePassport

基于欧洲电子护照机读区（MRZ）信息生成 AES 密钥，使用 CBC 模式解密给定密文。核心代码为 ans.py，涉及 MRZ 信息提取、SHA1 哈希密钥生成、奇偶校验处理及 CBC 解密流程。

过程

1. Byte-at-a-time ECB decryption (Harder)

(1)确定块大小

向加密接口输入长度递增的 b'X'*i，观察密文长度变化。当长度增加时，增量即为块大小（实验中为 16 字节）。核心代码(6.py):

```
previous_length = len(oracle.encrypt(b''))
for i in range(20):
    length = len(oracle.encrypt(b'X'*i))
    if length != previous_length:
        block_size = length - previous_length  # 结果为 16
        break
```

(2)确定前缀长度

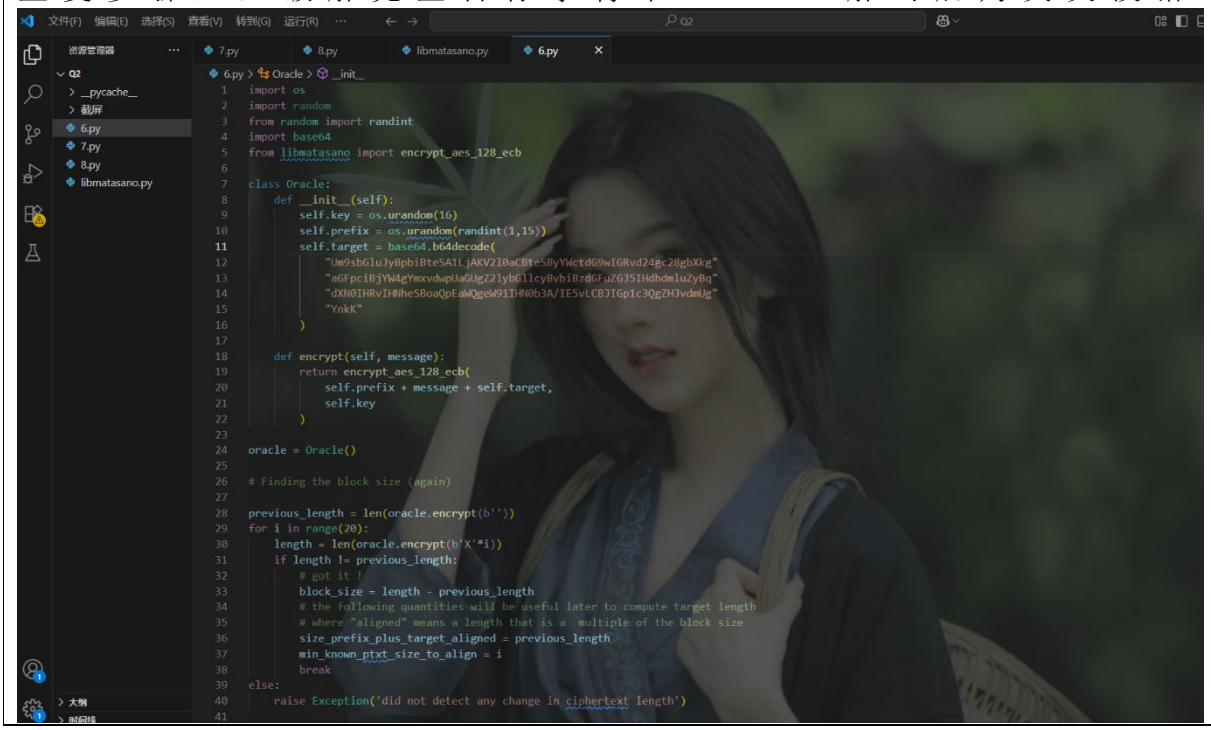
对比不同长度 b'X'*i 的密文块，当首块稳定时，计算前缀长度 prefix_size = block_size - i + 1。例如，当前缀长度为 5 时，输入 b'X'*11 可使前缀 + 填充恰好填满一个块。

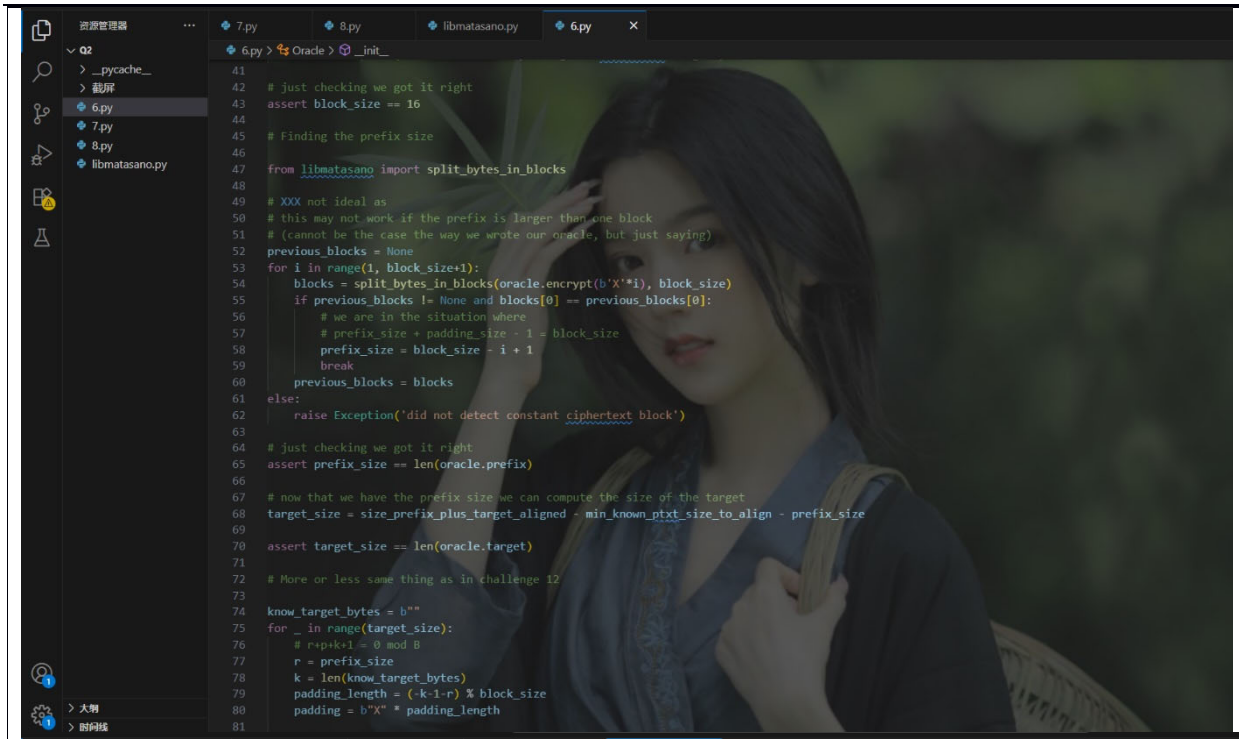
(3)逐字节破解目标字符串

构造填充 padding，使前缀+padding+已知字节+未知字节恰好对齐为整数块；

枚举所有可能字节（0-255），对比加密后目标块与预设块，匹配则确定该字节；

重复步骤直至破解完整目标字符串（Base64 解码后为英文段落）。





```
# just checking we got it right
assert block_size == 16

# Finding the prefix size

from libmatasano import split_bytes_in_blocks

# XXX not ideal as
# this may not work if the prefix is larger than one block
# (cannot be the case the way we wrote our oracle, but just saying)
previous_blocks = None
for i in range(1, block_size+1):
    blocks = split_bytes_in_blocks(oracle.encrypt(b'X'*i), block_size)
    if previous_blocks != None and blocks[0] == previous_blocks[0]:
        # we are in the situation where
        # prefix_size + padding_size - 1 = block_size
        prefix_size = block_size - i + 1
        break
    previous_blocks = blocks
else:
    raise Exception('did not detect constant ciphertext block')

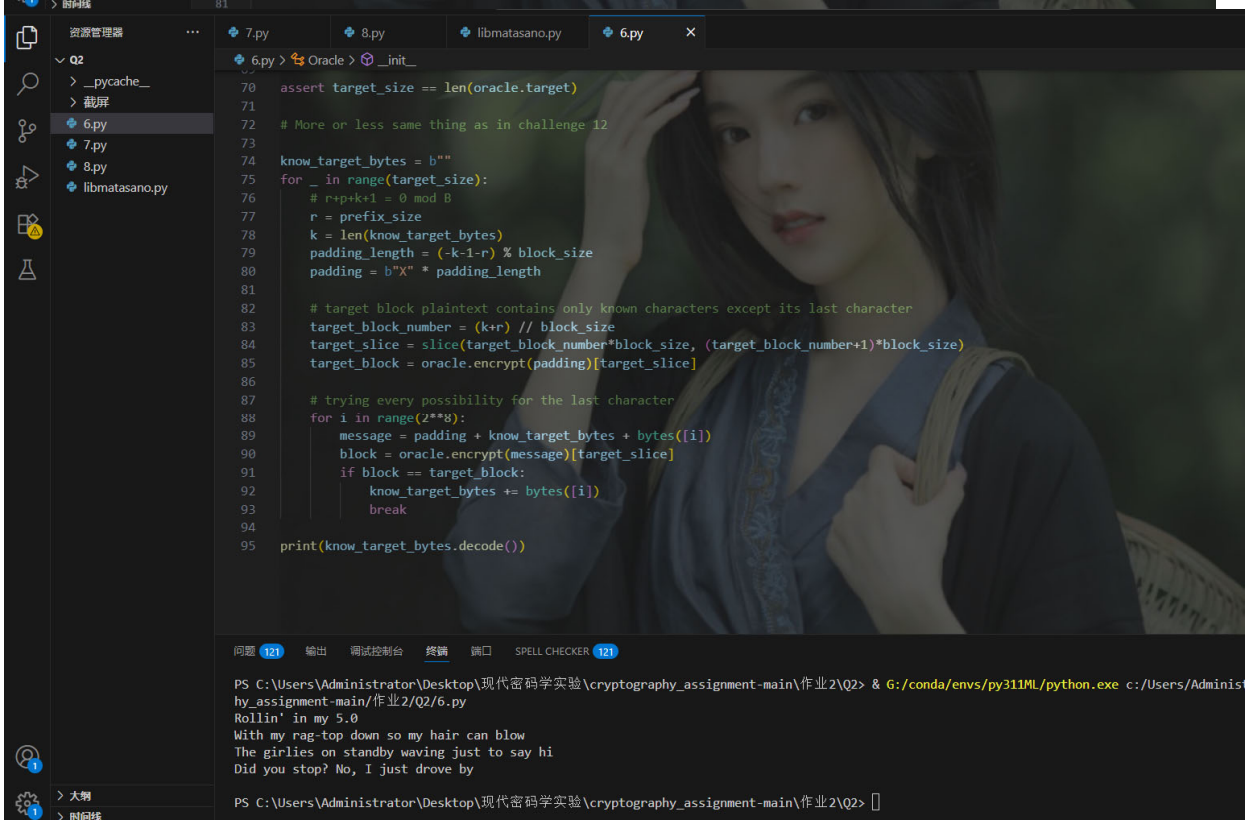
# just checking we got it right
assert prefix_size == len(oracle.prefix)

# now that we have the prefix size we can compute the size of the target
target_size = size_prefix_plus_target_aligned - min_known_ptxt_size_to_align - prefix_size

assert target_size == len(oracle.target)

# More or less same thing as in challenge 12

know_target_bytes = b""
for _ in range(target_size):
    # r+p+k+1 = 0 mod B
    r = prefix_size
    k = len(know_target_bytes)
    padding_length = (-k-1-r) % block_size
    padding = b"X" * padding_length
```



```
assert target_size == len(oracle.target)

# More or less same thing as in challenge 12

know_target_bytes = b""
for _ in range(target_size):
    # r+p+k+1 = 0 mod B
    r = prefix_size
    k = len(know_target_bytes)
    padding_length = (-k-1-r) % block_size
    padding = b"X" * padding_length

    # target block plaintext contains only known characters except its last character
    target_block_number = (k+r) // block_size
    target_slice = slice(target_block_number*block_size, (target_block_number+1)*block_size)
    target_block = oracle.encrypt(padding)[target_slice]

    # trying every possibility for the last character
    for i in range(2**8):
        message = padding + know_target_bytes + bytes([i])
        block = oracle.encrypt(message)[target_slice]
        if block == target_block:
            know_target_bytes += bytes([i])
            break

print(know_target_bytes.decode())
```

```
PS C:\Users\Administrator\Desktop\现代密码学实验\cryptography_assignment-main\作业2\Q2> & G:/conda/envs/py311ML/python.exe c:/Users/Administ
hy_assignment-main/作业2/Q2/6.py
Rollin' in my 5.0
With my rag-top down so my hair can blow
The girlies on standby waving just to say hi
Did you stop? No, I just drove by

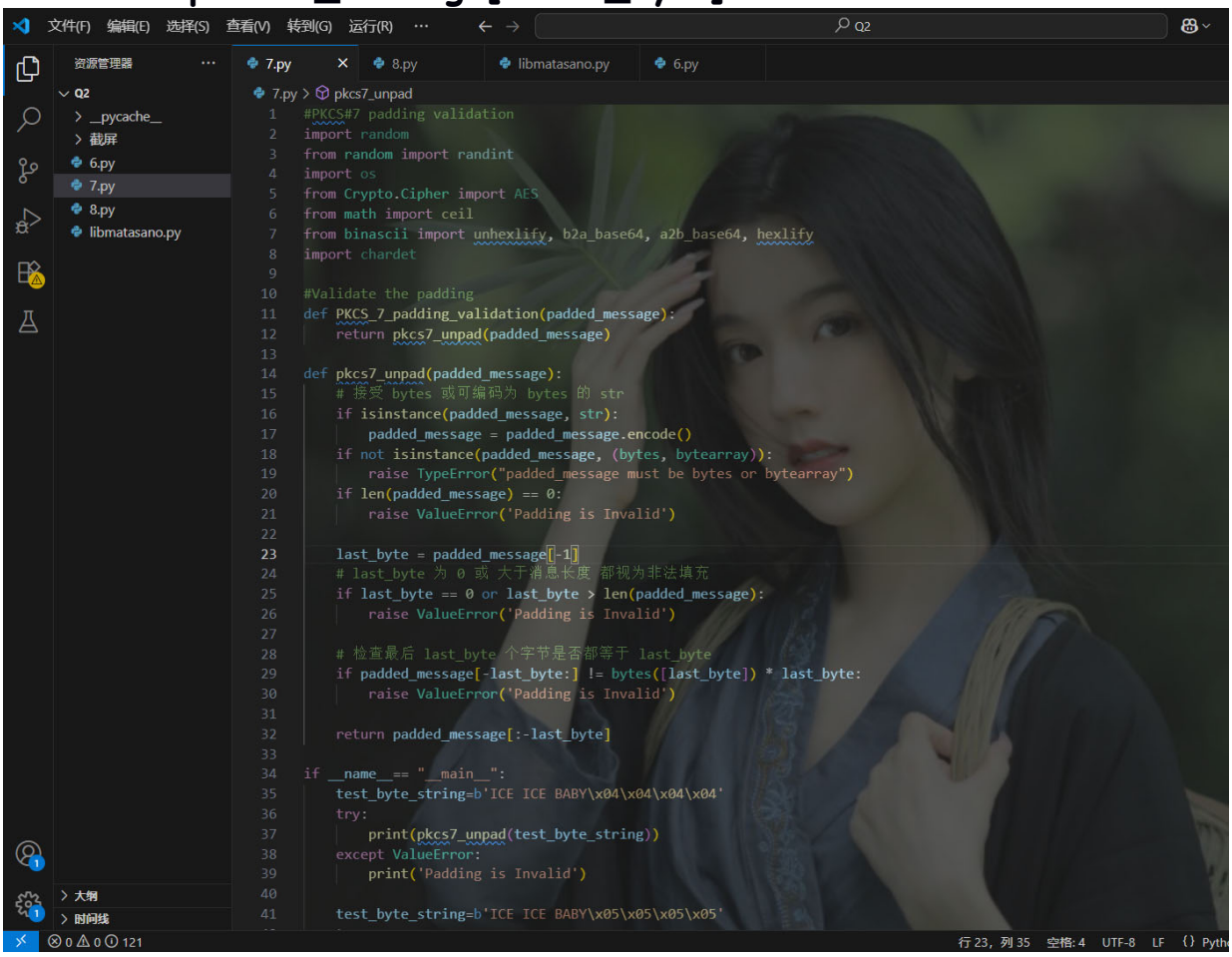
PS C:\Users\Administrator\Desktop\现代密码学实验\cryptography_assignment-main\作业2\Q2>
```

2. PKCS#7 padding validation

(1)验证逻辑实现：

```
def pkcs7_unpad(padded_message):
    if len(padded_message) == 0:
        raise ValueError('Invalid padding')
```

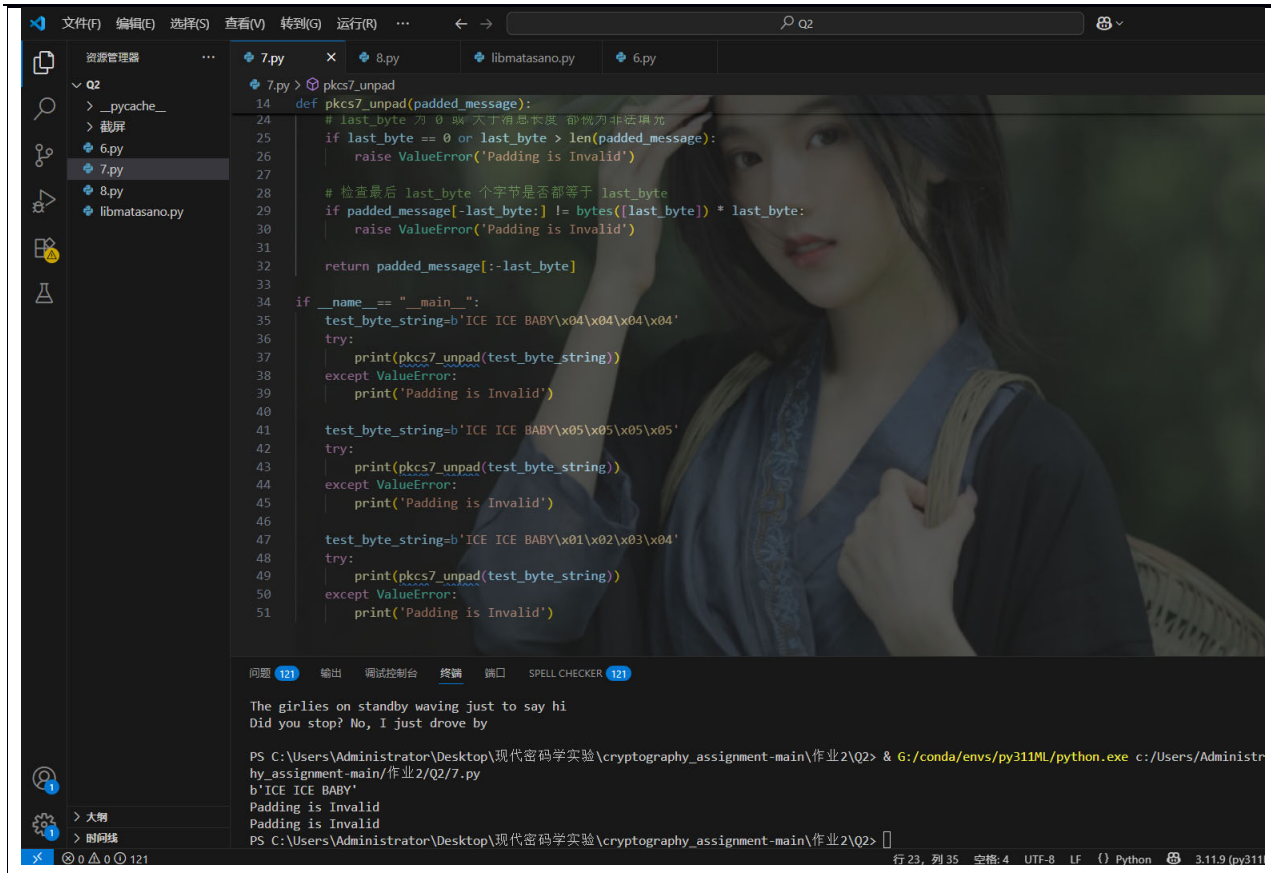
```
    last_byte = padded_message[-1]
    if last_byte == 0 or last_byte > len(padded_message):
        raise ValueError('Invalid padding')
    if padded_message[-last_byte:] != bytes([last_byte]) * last_byte:
        raise ValueError('Invalid padding')
    return padded_message[:-last_byte]
```



(2)测试案例：

有效填充：b'ICE ICE BABY\x04\x04\x04\x04' → 成功去除填充，返回 b'ICE ICE BABY'。

无效填充：b'ICE ICE BABY\x05\x05\x05\x05'（填充长度超实际）、b'ICE ICE BABY\x01\x02\x03\x04'（填充字节不一致）均抛出异常。



3. CBC bit flipping attacks

(1)确定前缀块数：对比空明文与单字节明文的密文前缀，计算前缀占用的块数 prefix_block_size（实验中为 1 块）。

(2)构造攻击明文与翻转比特：

输入 b'xadminxtruex'，使加密后目标明文位于特定块。

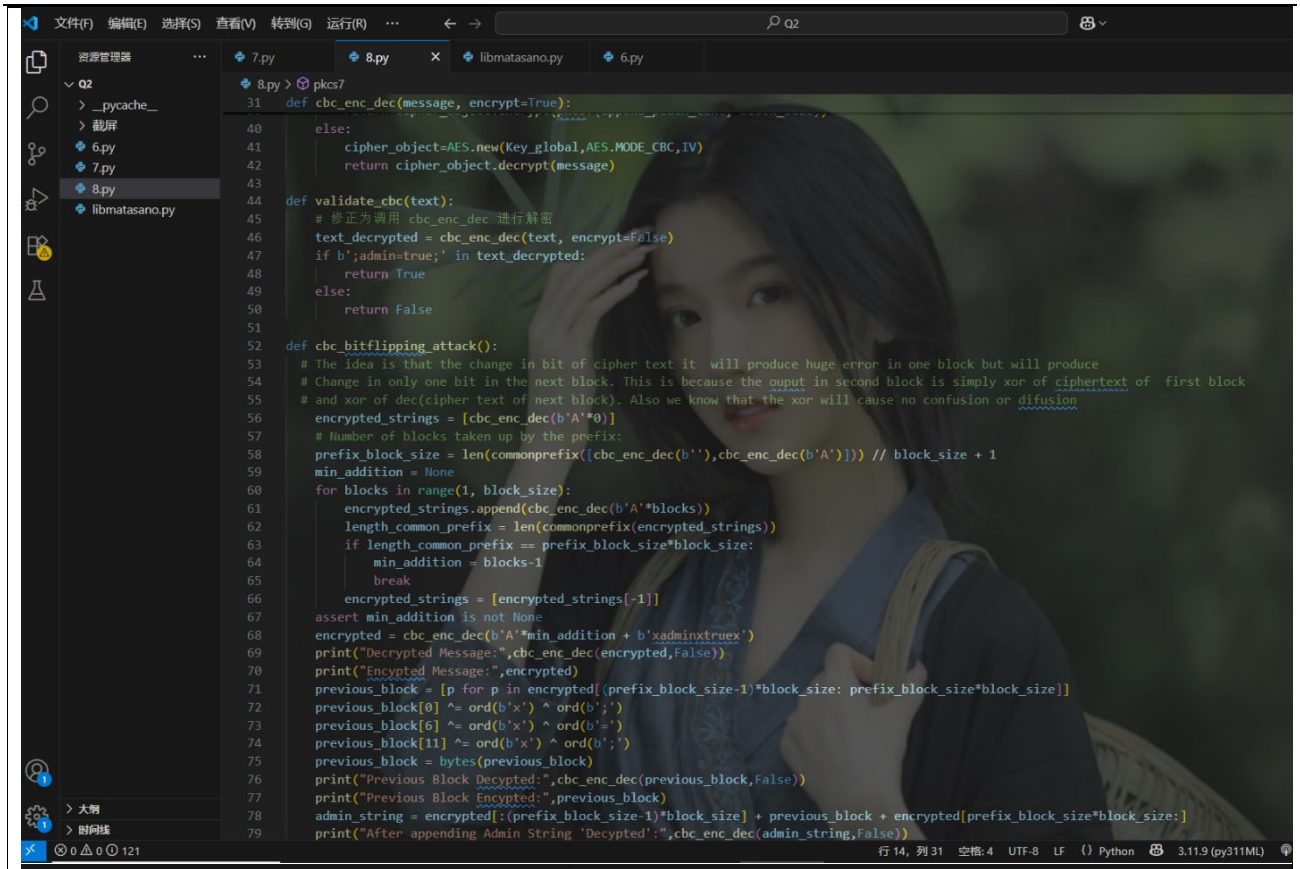
翻转前序密文块中对应位置的比特：

将第 0 位 x 翻转为;（previous_block[0] ^= ord(b'x') ^ ord(b';')）。

将第 6 位 x 翻转为=（previous_block[6] ^= ord(b'x') ^ ord(b'=')）。

将第 11 位 x 翻转为;（previous_block[11] ^= ord(b'x') ^ ord(b';')）。

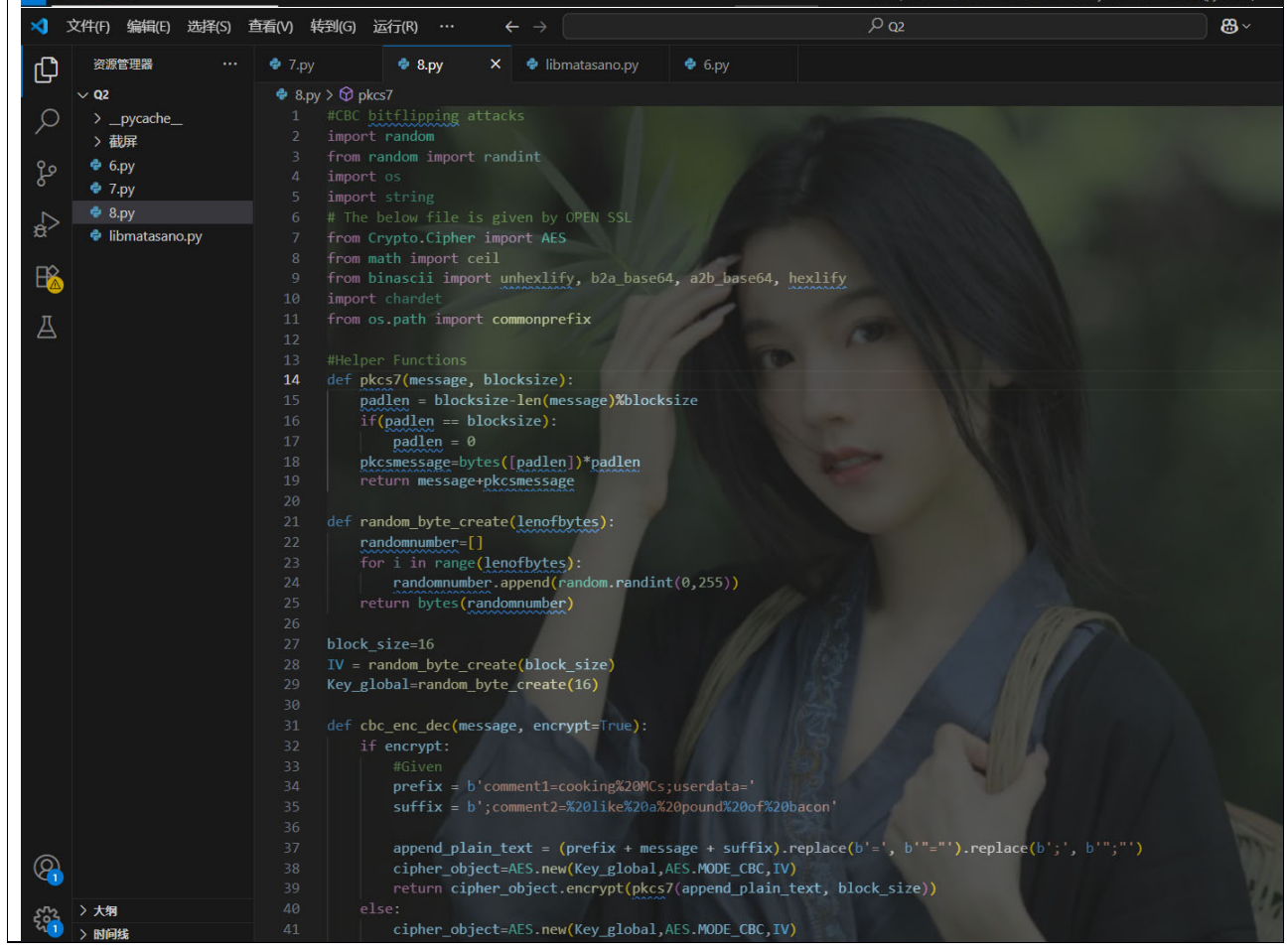
(3)验证攻击结果：解密修改后的密文，检查是否包含;admin=true;，实验返回 True，攻击成功。



```python
def cbc_enc_dec(message, encrypt=True):
    else:
        cipher_object=AES.new(Key_global,AES.MODE_CBC,IV)
        return cipher_object.decrypt(message)

def validate_cbc(text):
    # 修正为调用 cbc_enc_dec 进行解密
    text_decrypted = cbc_enc_dec(text, encrypt=False)
    if b';admin=true;' in text_decrypted:
        return True
    else:
        return False

def cbc_bitflipping_attack():
  # The idea is that the change in bit of cipher text it  will produce huge error in one block but will produce
  # Change in only one bit in the next block. This is because the ouput in second block is simply xor of ciphertext of  first block
  # and xor of dec(cipher text of next block). Also we know that the xor will cause no confusion or difusion
    encrypted_strings = [cbc_enc_dec(b'A'*0)]
    # Number of blocks taken up by the prefix:
    prefix_block_size = len(commonprefix([cbc_enc_dec(b''),cbc_enc_dec(b'A')])) // block_size + 1
    min_addition = None
    for blocks in range(1, block_size):
        encrypted_strings.append(cbc_enc_dec(b'A'*blocks))
        length_common_prefix = len(commonprefix(encrypted_strings))
        if length_common_prefix == prefix_block_size*block_size:
            min_addition = blocks-1
            break
        encrypted_strings = [encrypted_strings[-1]]
    assert min_addition is not None
    encrypted = cbc_enc_dec(b'A'*min_addition + b'xadminxtruex')
    print("Decrypted Message:",cbc_enc_dec(encrypted,False))
    print("Encypted Message:",encrypted)
    previous_block = [p for p in encrypted[(prefix_block_size-1)*block_size: prefix_block_size*block_size]]
    previous_block[0] ^= ord(b'x') ^ ord(b';')
    previous_block[6] ^= ord(b'x') ^ ord(b'=')
    previous_block[11] ^= ord(b'x') ^ ord(b';')
    previous_block = bytes(previous_block)
    print("Previous Block Decypted:",cbc_enc_dec(previous_block,False))
    print("Previous Block Encypted:",previous_block)
    admin_string = encrypted[:(prefix_block_size-1)*block_size] + previous_block + encrypted[prefix_block_size*block_size:]
    print("After appending Admin String 'Decypted':",cbc_enc_dec(admin_string,False))
```



```python
#CBC bitflipping attacks
import random
from random import randint
import os
import string
# The below file is given by OPEN SSL
from Crypto.Cipher import AES
from math import ceil
from binascii import unhexlify, b2a_base64, a2b_base64, hexlify
import chardet
from os.path import commonprefix

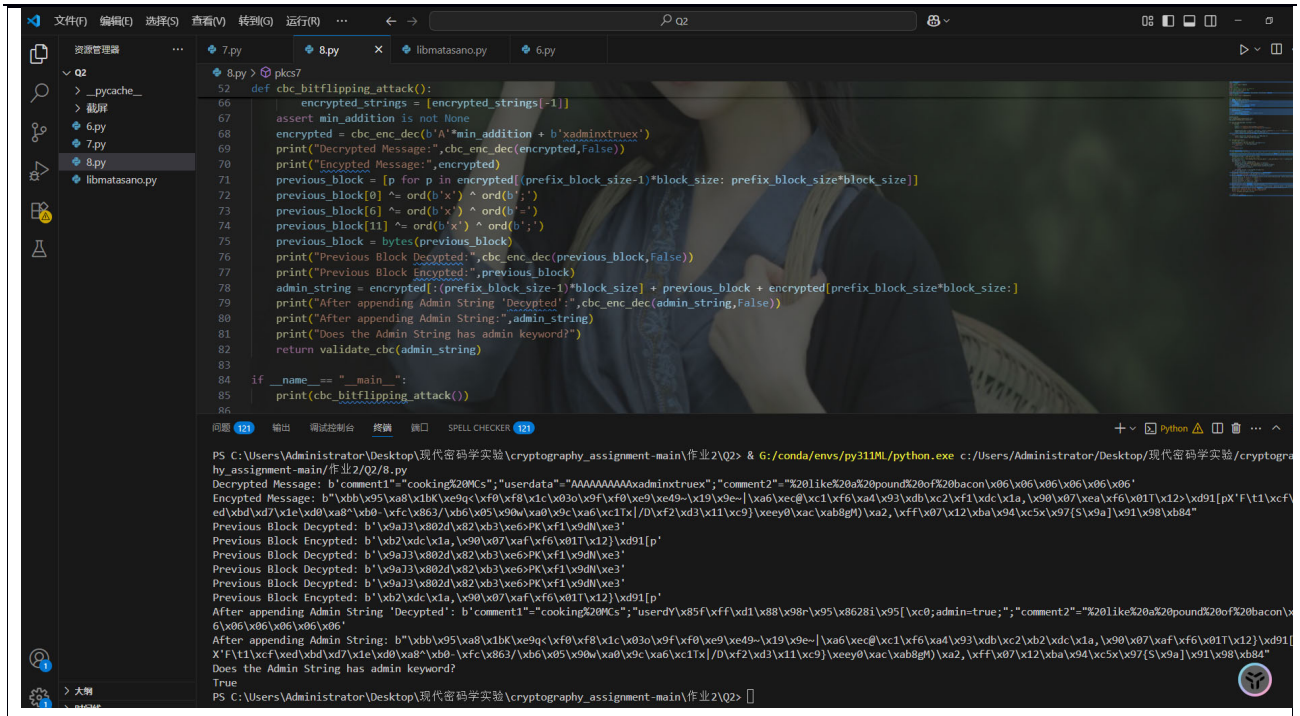
#Helper Functions
def pkcs7(message, blocksize):
    padlen = blocksize-len(message)%blocksize
    if(padlen == blocksize):
        padlen = 0
    pkcsmessage=bytes([padlen])*padlen
    return message+pkcsmessage

def random_byte_create(lenofbytes):
    randomnumber=[]
    for i in range(lenofbytes):
        randomnumber.append(random.randint(0,255))
    return bytes(randomnumber)

block_size=16
IV = random_byte_create(block_size)
Key_global=random_byte_create(16)

def cbc_enc_dec(message, encrypt=True):
    if encrypt:
        #Given
        prefix = b'comment1=cooking%20MCs;userdata='
        suffix = b';comment2=%20like%20a%20pound%20of%20bacon'

        append_plain_text = (prefix + message + suffix).replace(b'=', b'"="').replace(b';', b'";"')
        cipher_object=AES.new(Key_global,AES.MODE_CBC,IV)
        return cipher_object.encrypt(pkcs7(append_plain_text, block_size))
    else:
        cipher_object=AES.new(Key_global,AES.MODE_CBC,IV)
```



4. 2MTC3 AES key 生成与解密

(1)MRZ 信息提取：从输入字符串 v 中提取编号（no）、生日（birth）、日期（date），拼接为 mrz_information（如'12345678<811101821111167'）。

(2)密钥生成流程：

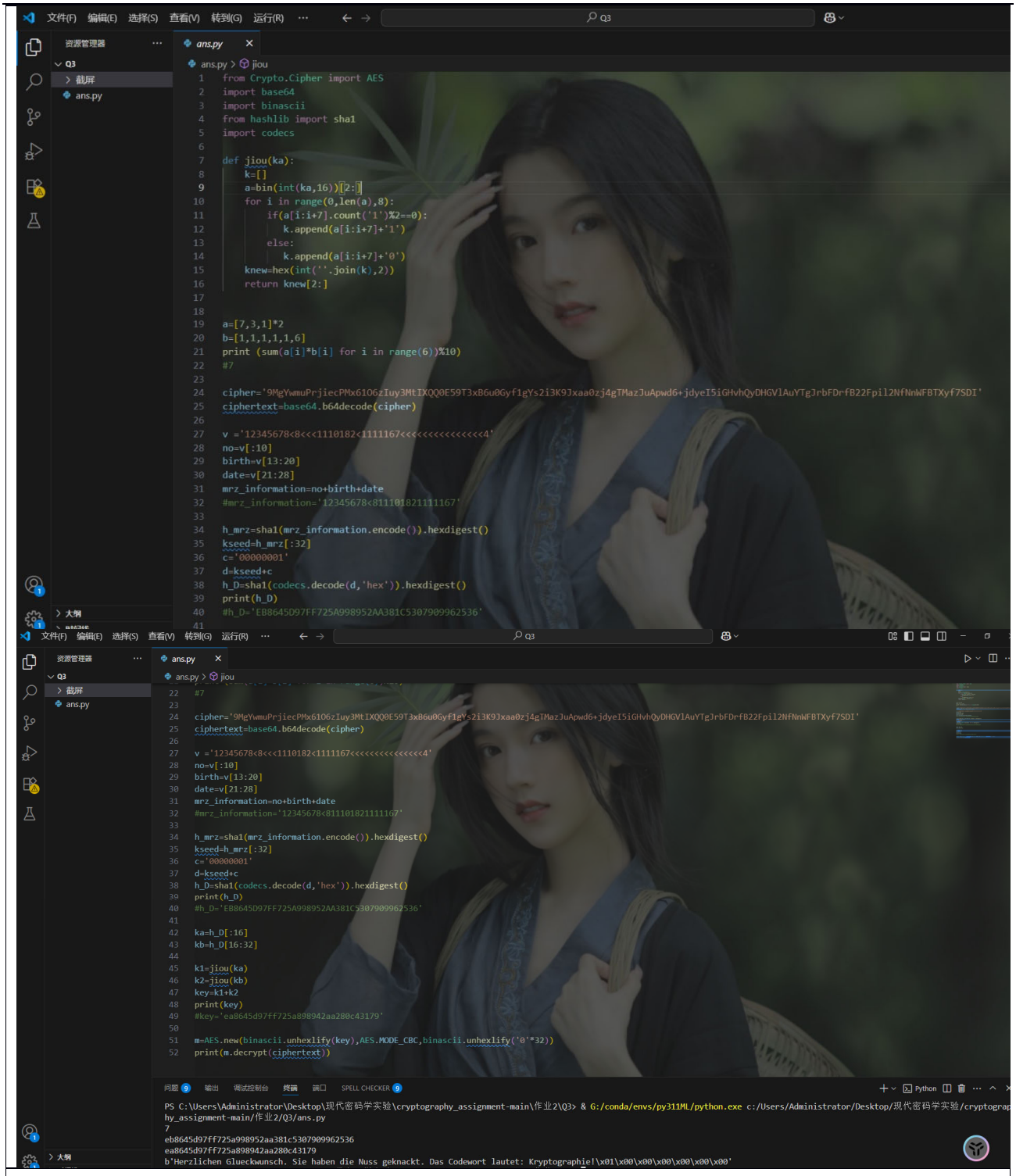
对 mrz_information 进行 SHA1 哈希，取前 32 位作为 kseed。

拼接 kseed 与 00000001 后再次 SHA1 哈希，得到 h_D（如 'EB8645D97FF725A998952AA381C5307909962536'）。

对 h_D 的前 16 位（ka）和后 16 位（kb）进行奇偶校验处理（jiou 函数），生成最终密钥 key（如'ea8645d97ff725a898942aa280c43179'）。

(3)CBC 解密：使用生成的密钥和全零 IV 解密 Base64 编码的密文，核心代码：

```
m = AES.new(binascii.unhexlify(key), AES.MODE_CBC,
binascii.unhexlify('0'*32))
print(m.decrypt(ciphertext))
# 输出解密后的明文
```



```python
from Crypto.Cipher import AES
import base64
import binascii
from hashlib import sha1
import codecs

def jiou(ka):
    k=[]
    a=bin(int(ka,16))[2:]
    for i in range(0,len(a),8):
        if(a[i:i+7].count('1')%2==0):
            k.append(a[i:i+7]+'1')
        else:
            k.append(a[i:i+7]+'0')
    knew=hex(int(''.join(k),2))
    return knew[2:]


a=[7,3,1]*2
b=[1,1,1,1,1,6]
print (sum(a[i]*b[i] for i in range(6))%10)
#7

cipher='9MgYwmuPrjiecPMx610GzIuy3MtIXQQ0E59T3xB6u0Gyf1gYs2i3K9Jxaa0zj4gTMazJuApwd6+jdyeI5iGHvhQyDHGVlAuYTgJrbFDrfB22Fpil2NfNnWFBTXyf7SDI'
ciphertext=base64.b64decode(cipher)

v ='12345678<8<<<1110182<1111167<<<<<<<<<<<<<<4'
no=v[:10]
birth=v[13:20]
date=v[21:28]
mrz_information=no+birth+date
#mrz_information='12345678<811101821111167'

h_mrz=sha1(mrz_information.encode()).hexdigest()
kseed=h_mrz[:32]
c='00000001'
d=kseed+c
h_D=sha1(codecs.decode(d,'hex')).hexdigest()
print(h_D)
#h_D='EB8645D97FF725A998952AA381C5307909962536'
```

```python
#7

cipher='9MgYwmuPrjiecPMx610GzIuy3MtIXQQ0E59T3xB6u0Gyf1gYs2i3K9Jxaa0zj4gTMazJuApwd6+jdyeI5iGHvhQyDHGVlAuYTgJrbFDrfB22Fpil2NfNnWFBTXyf7SDI'
ciphertext=base64.b64decode(cipher)

v ='12345678<8<<<1110182<1111167<<<<<<<<<<<<<<4'
no=v[:10]
birth=v[13:20]
date=v[21:28]
mrz_information=no+birth+date
#mrz_information='12345678<811101821111167'

h_mrz=sha1(mrz_information.encode()).hexdigest()
kseed=h_mrz[:32]
c='00000001'
d=kseed+c
h_D=sha1(codecs.decode(d,'hex')).hexdigest()
print(h_D)
#h_D='EB8645D97FF725A998952AA381C5307909962536'

ka=h_D[:16]
kb=h_D[16:32]

k1=jiou(ka)
k2=jiou(kb)
key=k1+k2
print(key)
#key='ea8645d97ff725a898942aa280c43179'

m=AES.new(binascii.unhexlify(key),AES.MODE_CBC,binascii.unhexlify('0'*32))
print(m.decrypt(ciphertext))
```

```
PS C:\Users\Administrator\Desktop\现代密码学实验\cryptography_assignment-main\作业2\Q3> & G:/conda/envs/py311ML/python.exe c:/Users/Administrator/Desktop/现代密码学实验/cryptography_assignment-main/作业2/Q3/ans.py
7
eb8645d97ff725a998952aa381c5307909962536
ea8645d97ff725a898942aa280c43179
b'Herzlichen Glueckwunsch. Sie haben die Nuss geknackt. Das Codewort lautet: Kryptographie!\x01\x00\x00\x00\x00\x00\x00'
```

总结

ECB 模式安全性： ECB 模式因确定性加密易受逐字节攻击，随机前缀仅增加攻击复杂度（需先对齐块），但无法根本解决安全问题，实际中应避免使用。

PKCS#7 填充的重要性： 填充验证是对称加密的基础环节，不当的验证逻辑可能导致 Padding Oracle 攻击，需严格校验填充长度与内容的一致性。

CBC 模式的风险： CBC 模式的比特翻转攻击利用前序密文块对后续明文的影响，提示需确保 IV 随机性，并优先选择带认证的加密模式（如 GCM）。

密钥生成规范： 基于 MRZ 的密钥生成依赖哈希函数（如 SHA1）和奇偶校验，虽满足特定场景需求，但 SHA1 安全性已受质疑，实际应采用 SHA-256 等更强算法，并确保密钥生成过程的随机性。

参考文献

无